

DSA Theory (combined)

All 13 pattern files in roadmap order, combined into one document. The source files are each folder's `theory.md`. If you edit one, regenerate this file rather than editing it here.

Contents

1. Binary search, in full (`binarySearch/`)
2. Hash map, in full (`hashmap/`)
3. Two pointers, in full (`twoPointers/`)
4. Sliding window, in full (`slidingWindow/`)
5. Prefix sum, in full (`prefixSum/`)
6. Stack, in full (`stack/`)
7. Intervals, in full (`intervals/`)
8. Linked list, in full (`linkedList/`)
9. Trees (DFS and BFS), in full (`trees/`)
10. Heap / priority queue, in full (`heap/`)
11. Backtracking, in full (`backtracking/`)
12. Graphs (BFS, DFS, topological sort), in full (`graphs/`)
13. Dynamic programming (1-D), in full (`dp/`)

Binary search, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
 - Invariant:
 - Template/pseudocode from memory:
 - Complexity:
 - Solved/re-solved: 704, 35, 34 (accepted 2026-09-26/27)
 - Mistake and correction:
-

One word: Halving. **One sentence:** Binary search keeps a window that is guaranteed to contain the answer and throws away half of it on every step, so it finishes in $O(\log n)$.

Executive summary

- **The window:** `lo` and `hi` bound where the answer can still be. `mid` is the probe.
 - **The invariant:** "if the answer exists, it is inside the window." Every move must keep this true.
 - **Why $O(\log n)$:** the window size goes $n, n/2, n/4 \dots 1$. That is about $\log_2 n$ steps, so 10^9 items takes about 30 probes.
 - **What you modify:** four knobs. The window style (closed or half-open), the loop condition, how `lo` / `hi` move, and what you return. Each knob follows from the question you ask at `mid`.
 - **Two pointers is different:** pointers step by one, so it is $O(n)$. Use it for pairs, palindromes and partitions.
-

1. The core template (exact match)

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1      # closed window [lo, hi]
    while lo <= hi:                # window non-empty
        mid = lo + (hi - lo) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            lo = mid + 1            # mid and everything left is too small
        else:
            hi = mid - 1           # mid and everything right is too big
    return -1
```

Trace on `[1, 3, 5, 7, 9, 11]`, target 9:

step	lo	hi	mid	nums[mid]	action
1	0	5	2	5	5 < 9, lo = 3
2	3	5	4	9	found, return 4

Target 4 (missing): (0,5,mid 2 $\rightarrow$ 5>4, hi=1), (0,1,mid 0 $\rightarrow$ 1<4, lo=1), (1,1,mid 1 $\rightarrow$ 3<4, lo=2). Now lo=2 > hi=1, the window is empty, return -1. Notice lo stopped at 2, which is exactly where 4 would be inserted. That fact powers the lower-bound variant.

2. Why it is correct

- **Initialization:** the window is the whole array, so the invariant holds.
- **Maintenance:** the array is sorted. If `nums[mid] < target`, every index $\leq$ mid is also < target, so dropping them loses nothing. Same logic on the right.
- **Progress:** each step removes at least `mid` itself, so the window strictly shrinks. No infinite loop.
- **Termination:** the loop ends when the window is empty. By the invariant, the target is not in the array.

3. Why $O(\log n)$, precisely

After k steps the window has at most $n / 2^k$ elements. It becomes empty once $2^k > n$, so $k \approx \lfloor \log_2 n \rfloor + 1$. Space is $O(1)$ iteratively, $O(\log n)$ if written recursively. It matches the comparison lower bound: any comparison search needs $\Omega(\log n)$ probes, because a decision tree with n leaves has depth at least $\log_2 n$.

4. The four knobs, and when to change each

Knob A: closed $[lo, hi]$ vs half-open $[lo, hi)$

- **Closed** ($hi = n - 1$): use when you are looking for an exact element and may return from inside the loop.
- **Half-open** ($hi = n$): use when the answer may be "one past the end", like an insertion point. $hi = n$ is a legal answer.
- **Rule:** the initial hi defines the window, and every other knob must agree with it.

Knob B: while $lo \leq hi$ VS while $lo < hi$

- $lo \leq hi$ goes with closed windows. The loop runs while at least one element remains, and you exit with $lo = hi + 1$.
- $lo < hi$ goes with "shrink until one candidate remains". You exit with $lo == hi$, which is the answer. Use it for lower bound, peak finding and minimum in a rotated array.
- **Bug:** pairing $lo < hi$ with $hi = mid - 1$ skips checking the last element.

Knob C: how lo and hi move

Ask: "can mid itself still be the answer?"

- **No, mid is ruled out** → $lo = mid + 1$ or $hi = mid - 1$.
- **Yes, mid may be the answer** → keep it: $hi = mid$ (answer is at mid or left) or $lo = mid$ (answer is at mid or right).
- **Danger with $lo = mid$:** when $hi = lo + 1$, the default mid equals lo and the loop never moves. Fix it by rounding mid up: $mid = lo + (hi - lo + 1) // 2$.
- **Rule of thumb:** $hi = mid$ pairs with the lower mid , $lo = mid$ pairs with the upper mid .

Knob D: what you return

- Exact match: return inside the loop, else -1 .
- First/last position or insertion point: return lo after the loop, then check it is in range and holds the target if the question needs that.

5. The variants you will actually meet

Lower bound: first index with `nums[i] >= target` (same as `bisect_left`)

```
def lower_bound(nums, target):
    lo, hi = 0, len(nums)          # half-open, n is a valid answer
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] < target:
            lo = mid + 1           # mid too small, rule it out
        else:
            hi = mid               # mid could be the answer, keep it
    return lo
```

Invariant: everything left of `lo` is $<$ target, everything at or right of `hi` is $\geq$ target.

Upper bound: first index with `nums[i] > target` (same as `bisect_right`). Change only `<` to `<=` in the comparison. The count of `target` is `upper - lower`, and the last occurrence is `upper - 1`.

Last index satisfying a condition (upper mid):

```
lo, hi = 0, n - 1
while lo < hi:
    mid = lo + (hi - lo + 1) // 2  # round up because of lo = mid
    if ok(mid): lo = mid
    else:      hi = mid - 1
```

Binary search on the answer. This is the most common Google form. The array is not searched at all. You search a range of possible answers where `feasible(x)` flips from False to True exactly once. Examples: Koko eating bananas, ship within D days, split array largest sum.

```
def min_feasible(lo, hi, feasible):
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if feasible(mid): hi = mid  # mid works, maybe something smaller does
        else:             lo = mid + 1
    return lo
```

Cost is $O(\log(\text{range}) \times \text{cost of feasible})$.

Rotated sorted array. At every mid, one half is sorted. Check whether the target falls in the sorted half. If yes, search there, else go to the other half.

Peak element / minimum in rotated array. Compare `nums[mid]` with `nums[mid + 1]` or with `nums[hi]` instead of with a target. Use the `lo < hi` form with `hi = mid`.

6. Decision table: which knobs for which question

Question	Window	Loop	Moves	Return
Does x exist?	[0, n-1]	lo <= hi	±1 both sides	mid or -1
First ≥ x / insert position	[0, n)	lo < hi	lo=mid+1, hi=mid	lo
First > x	[0, n)	lo < hi	same, with <=	lo
Last satisfying ok	[0, n-1]	lo < hi	lo=mid, hi=mid-1, upper mid	lo
Minimum feasible answer	[min, max]	lo < hi	hi=mid, lo=mid+1	lo

7. Common bugs

1. `(lo + hi) // 2` overflows in Java/C++. Python ints don't overflow, but write `lo + (hi - lo) // 2` in interviews to show you know.
2. Mixing a closed window with `lo < hi`, which skips an element.
3. `lo = mid` with the lower mid, which loops forever.
4. Forgetting that lower bound can return `n`. Check `lo < n and nums[lo] == target`.
5. Using binary search on data that is not monotonic.

8. Two pointers, for contrast

```
def pair_sum(nums, k):
    # nums sorted
    i, j = 0, len(nums) - 1
    while i < j:
        s = nums[i] + nums[j]
        if s == k: return i, j
        if s < k: i += 1
        else: j -= 1
    return None
```

need bigger, drop smallest
need smaller, drop largest

Invariant: no valid pair uses an index outside `[i, j]`. Each step drops one element, not half, so it is $O(n)$. Choose binary search when you ask "where does a monotonic condition flip?" Choose two pointers when you look at pairs or a window that moves one step at a time.

9. Interview prep

- **Control flow:** initialize the window, loop while it has candidates, probe mid, move one boundary according to the monotonic test, then return from inside the loop or return `lo`.
 - **Purpose:** find where a monotonic predicate changes from False to True, in an array or over a range of answers.
 - **Output:** an index, an insertion point, or the smallest feasible value.
 - **Trade-offs:** $O(\log n)$ time and $O(1)$ space, but it needs sorted or monotonic input. Sorting first costs $O(n \log n)$, so for one lookup a linear scan is cheaper. For many lookups, sort once and search many times. In production Python, use `bisect`.
 - **What a strong candidate says:** "I'll define the predicate first. Here it is `nums[i] >= target`, which is False then True. I'll use a half-open window `[0, n)` because `n` is a valid answer. The invariant is that left of `lo` is False and `hi` onward is True. At mid, if the predicate is False I set `lo = mid + 1`, otherwise `hi = mid` since mid might be the answer. The window shrinks every step, so it ends in $O(\log n)$ with `lo == hi` at the first True."
 - **Likely Google follow-ups:** first and last position of a target, search a rotated array, find the minimum of a rotated array, median of two sorted arrays, and "binary search on the answer" problems.
-

Part 2: going deeper

10. How to recognize a binary search problem

Reach for binary search when you see any of these:

- **Sorted input** (or "non-decreasing", "rotated sorted") and a question about position, existence or count.
- **"Minimum X such that..." / "maximum X such that..."**: the smallest speed, capacity, day or size that works. That is binary search on the answer.
- **A yes/no check that flips once**. If `feasible(x)` is True, then `feasible(x + 1)` is also True (or the reverse). That is called monotonic.
- **Constraints**: n up to 10^5 – 10^6 with values up to 10^9 , and a hint that $O(n^2)$ is too slow. $O(n \log n)$ or $O(n \log V)$ is the target.
- **"Find a peak", "find the rotation point"**: no target at all, but you can still tell which half to keep.

Ask yourself one question: **"If I probe the middle, can I throw away one half for sure?"** If yes, it's binary search.

11. More traces

Leftmost position (lower bound) with duplicates

`nums = [5, 7, 7, 8, 8, 10]`, target 8, half-open window `[0, 6)`:

step	lo	hi	mid	nums[mid]	nums[mid] < 8?	action
1	0	6	3	8	no	hi = 3 (mid might be the first 8)
2	0	3	1	7	yes	lo = 2
3	2	3	2	7	yes	lo = 3
end	3	3				return 3

Rightmost position (upper bound – 1)

Same array, "first index with `nums[i] > 8`":

step	lo	hi	mid	nums[mid]	nums[mid] <= 8?	action
1	0	6	3	8	yes	lo = 4
2	4	6	5	10	no	hi = 5
3	4	5	4	8	yes	lo = 5
end	5	5				upper = 5, last 8 is at 4

LC 34 in terms of those two

```
def lower_bound(nums, target):
    lo, hi = 0, len(nums)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo

def searchRange(nums, target):
    first = lower_bound(nums, target)
    if first == len(nums) or nums[first] != target:
        return [-1, -1]
    last = lower_bound(nums, target + 1) - 1 # first index of anything bigger
    return [first, last]
```

`target + 1` works because the values are integers. For floats, write a separate upper bound with `<=`.

12. Worked problems

LC 33: Search in Rotated Sorted Array

Idea: a rotated array like `[4, 5, 6, 7, 0, 1, 2]` is two sorted runs. At any `mid`, at least one of the two halves is fully sorted. Check whether the target lies in the sorted half. If yes, go there, otherwise go to the other half.

```
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:           # left half [lo, mid] is sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                             # right half [mid, hi] is sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Trace `[4, 5, 6, 7, 0, 1, 2]`, target 0:

lo	hi	mid	nums[mid]	sorted half	target inside?	action
0	6	3	7	left ($4 \leq 7$)	$4 \leq 0 < 7$? no	lo = 4
4	6	5	1	left ($0 \leq 1$)	$0 \leq 0 < 1$? yes	hi = 4
4	4	4	0		found	return 4

Why `<=` in `nums[lo] <= nums[mid]`: when `lo == mid` (two elements left), the "left half" is one element, and one element is sorted. With `<` you would wrongly treat it as the unsorted side.

LC 153: Minimum in Rotated Sorted Array

```
def findMin(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] > nums[hi]:
            lo = mid + 1           # the drop is to the right of mid, mid is not the min
        else:
            hi = mid               # mid..hi is sorted, so the minimum is mid or the first element
    return nums[lo]
```

Trace `[3,4,5,1,2]`: (0,4, mid 2: $5 > 2 \rightarrow \text{lo} = 3$), (3,4, mid 3: $1 > 2$? no $\rightarrow \text{hi} = 3$), return `nums[3] = 1`. **Why compare with `nums[hi]` and not `nums[lo]`**: on an unrotated array `[1,2,3]`, `nums[mid] > nums[lo]` is True but the minimum is on the left. Comparing with `hi` works for both the rotated and the unrotated case.

LC 162: Find Peak Element

```
def findPeakElement(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if nums[mid] < nums[mid + 1]:
            lo = mid + 1      # going uphill to the right, so a peak exists c
        else:
            hi = mid          # downhill or flat to the right, so mid or some
    return lo
```

`mid + 1` never goes out of range, because `lo < hi` means `mid < hi`. The array isn't sorted, but the question "which side has a peak?" still has a sure answer. That's all binary search needs.

LC 74: Search a 2-D Matrix

Each row is sorted, and each row starts after the previous row ends, so the matrix is one sorted list of `m * n` values. Index `i` of that list is `matrix[i // n][i % n]`.

```
def searchMatrix(matrix, target):
    m, n = len(matrix), len(matrix[0])
    lo, hi = 0, m * n - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        v = matrix[mid // n][mid % n]
        if v == target:
            return True
        if v < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return False
```

LC 875: Koko Eating Bananas (binary search on the answer)

- **Answer range:** speed `k` from 1 to `max(piles)`. At `max(piles)`, every pile takes 1 hour, which always fits because `h >= len(piles)`.

- **Check:** `hours(k) = sum(ceil(p / k))`. A faster speed never takes more hours, so `hours(k) <= h` flips from False to True exactly once.
- **Goal:** the smallest `k` with `hours(k) <= h`, which is the "minimum feasible" template.

```
def minEatingSpeed(piles, h):
    def hours(k):
        return sum((p + k - 1) // k for p in piles) # integer ceil(p / k)
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if hours(mid) <= h:
            hi = mid # mid works, try slower
        else:
            lo = mid + 1 # too slow
    return lo
```

Trace `piles = [3, 6, 7, 11]`, `h = 8`:

lo	hi	mid	hours(mid)	fits?	action
1	11	6	1+1+2+2 = 6	yes	hi = 6
1	6	3	1+2+3+4 = 10	no	lo = 4
4	6	5	1+2+2+3 = 8	yes	hi = 5
4	5	4	1+2+2+3 = 8	yes	hi = 4
end					return 4

Cost: $O(n \cdot \log(\max(\text{piles})))$. `(p + k - 1) // k` is ceiling division without floats. `math.ceil(p / k)` also works but can hit float rounding on huge numbers.

13. Python's `bisect` module

```
import bisect
a = [1, 3, 3, 3, 7]
bisect.bisect_left(a, 3) # 1: first index with a[i] >= 3 (lower bound)
bisect.bisect_right(a, 3) # 4: first index with a[i] > 3 (upper bound)
bisect.bisect_right(a, 3) - bisect.bisect_left(a, 3) # 3: count of 3s
bisect.insort(a, 5) # inserts, keeping a sorted (the insert itself is
```

- Python 3.10+ accepts `key=` for searching by a derived value.

- **In an interview:** write the loop by hand unless they allow libraries. Then mention "in production I'd use `bisect_left`". That shows you know both.

14. Edge-case checklist

Run every binary search against these before submitting:

1. Empty array `[]`
2. One element, where the target equals it, is below it, and is above it
3. Two elements. Most off-by-one bugs show up here.
4. Target smaller than everything, and target larger than everything
5. All elements equal, e.g. `[2, 2, 2, 2]` with target 2, for first and last position
6. Target at index 0 and at index $n-1$

15. Debugging an off-by-one: the two-element trace

If you're unsure about a loop, trace it by hand on `[1, 3]` with targets 0, 1, 2, 3 and 4. That's five tiny traces. They catch:

- **Infinite loops:** `lo = mid` with the lower mid. On `lo = 0, hi = 1`, mid is 0 forever.
- **Skipped elements:** a closed window with `lo < hi` never checks the last candidate.
- **Out-of-range reads:** a half-open `hi = n` with `lo <= hi` reads `nums[n]`.

16. Recursive version, and why to prefer the loop

```
def search_rec(nums, target, lo=0, hi=None):
    if hi is None:
        hi = len(nums) - 1
    if lo > hi:
        return -1
    mid = lo + (hi - lo) // 2
    if nums[mid] == target:
        return mid
    if nums[mid] < target:
        return search_rec(nums, target, mid + 1, hi)
    return search_rec(nums, target, lo, mid - 1)
```

Same $O(\log n)$ time, but $O(\log n)$ stack space instead of $O(1)$. The loop is the standard answer. Recursion is useful mainly for explaining the idea.

17. Practice ladder

1. 704 Binary Search: exact match (done)
 2. 35 Search Insert Position: lower bound (done)
 3. 34 First and Last Position: lower and upper bound (done)
 4. 74 Search a 2-D Matrix: index mapping
 5. 153 Find Minimum in Rotated Sorted Array: compare with `hi`
 6. 33 Search in Rotated Sorted Array: find the sorted half
 7. 162 Find Peak Element: no target
 8. 875 Koko Eating Bananas: binary search on the answer
 9. 1011 Capacity to Ship Packages Within D Days: same as 875, with different bounds
-

Hash map, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved: 1, 242
- Mistake and correction:

One word: Lookup. **One sentence:** A hash map spends $O(n)$ extra memory so that "have I seen this before, and where?" costs $O(1)$ instead of a full scan.

Executive summary

- **What it is:** a table from key to value. `dict` in Python, `set` when you only need the keys.
- **Why it's fast:** the key is hashed to a bucket index, so lookup, insert and delete are $O(1)$ on average.
- **The trade:** $O(n)$ memory for $O(1)$ lookups. Most " $O(n^2) \rightarrow O(n)$ " improvements in interviews are this trade.
- **Five shapes you'll meet:** complement lookup, frequency count, grouping by a key, seen-set, and prefix sum plus a map.

1. How it works (just enough)

- `hash(key)` gives a number, and `number % capacity` picks a bucket.
- Two keys can land in the same bucket (a collision). Python probes other slots until it finds a free one.

- When the table gets about 2/3 full, Python grows it and re-inserts everything. That's $O(n)$ once in a while, so the cost averages out to $O(1)$ per insert.
- **Worst case is $O(n)$** per operation if many keys collide. It almost never happens with built-in types. Say " $O(1)$ average" in interviews.
- **Keys must be hashable**, meaning they can't change: `int`, `str` and `tuple` work, but `list`, `dict` and `set` don't. To use a list as a key, convert it: `tuple(lst)`.

2. Shape A: complement lookup (Two Sum, LC 1)

Question: find `i, j` with `nums[i] + nums[j] == target`. **Idea:** for each `x`, the partner you need is `target - x`. Remember every value you've passed and its index.

```
def twoSum(nums, target):
    seen = {} # value -> index
    for i, x in enumerate(nums):
        need = target - x
        if need in seen: # check BEFORE storing x
            return [seen[need], i]
        seen[x] = i
    return []
```

Trace `[2, 7, 11, 15]`, target 9:

i	x	need	seen before	action
0	2	7	{}	store 2 → 0
1	7	2	{2:0}	found, return [0, 1]

Invariant: at step `i`, `seen` holds every value from `nums[0..i-1]`, so any pair ending at `i` is found at step `i`. **Why check before storing:** with `[3]` and target 6, storing first would pair 3 with itself. With `[3, 3]`, checking first still finds `[0, 1]`.

3. Shape B: frequency count (Valid Anagram, LC 242)

```
from collections import Counter

def isAnagram(s, t):
    return Counter(s) == Counter(t)

# by hand, which interviewers sometimes ask for:
def isAnagram_manual(s, t):
    if len(s) != len(t):
        return False
    count = {}
    for a, b in zip(s, t):
        count[a] = count.get(a, 0) + 1
        count[b] = count.get(b, 0) - 1
    return all(v == 0 for v in count.values())
```

- `count.get(k, 0) + 1` is the standard counting idiom. `collections.defaultdict(int)` avoids the `.get`.
- If the input is only lowercase a–z, an array of 26 counts works too and is still $O(1)$ space.

4. Shape C: grouping by a key (Group Anagrams, LC 49)

Idea: give every word a key that's identical for all its anagrams, then group words by that key.

```
from collections import defaultdict

def groupAnagrams(strs):
    groups = defaultdict(list)
    for w in strs:
        key = [0] * 26
        for ch in w:
            key[ord(ch) - ord('a')] += 1
        groups[tuple(key)].append(w) # a list can't be a key, a tuple can
    return list(groups.values())
```

- The key could also be `''.join(sorted(w))`, which costs $O(k \log k)$ per word instead of $O(k)$ for the count tuple.
- **Total cost:** $O(n \cdot k)$, where k is the word length.

5. Shape D: seen-set (Contains Duplicate, LC 217; Longest Consecutive Sequence, LC 128)

```
def containsDuplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
# one-liner: return len(set(nums)) < len(nums)
```

LC 128 finds the longest run of consecutive integers in $O(n)$ without sorting:

```
def longestConsecutive(nums):
    s = set(nums)
    best = 0
    for x in s:
        if x - 1 not in s:          # x starts a run; only count from starts
            y = x
            while y + 1 in s:
                y += 1
            best = max(best, y - x + 1)
    return best
```

Why it's $O(n)$ with a nested loop: the inner `while` only runs from the start of a run, so each number is walked over at most once in total.

6. Shape E: top-k by frequency (LC 347), using bucket sort

```
from collections import Counter

def topKFrequent(nums, k):
    count = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)] # index = frequency
    for x, c in count.items():
        buckets[c].append(x)
    res = []
    for c in range(len(buckets) - 1, 0, -1):
        for x in buckets[c]:
            res.append(x)
            if len(res) == k:
                return res
    return res
```

$O(n)$, because a frequency can't be bigger than n . The heap version is $O(n \log k)$ (see [heap/theory.md](#)), and `Counter(nums).most_common(k)` does it in one line.

7. Shape F: prefix sum plus a map

"Count subarrays with sum k " uses a map from prefix sum to how many times it's been seen. Covered in full in [prefixSum/theory.md](#) (LC 560).

8. Decision table

Question	Structure	Key → value
Does a partner exist?	dict	value → index
Same letters / same counts?	Counter	item → count
Group things that are "the same"	defaultdict(list)	canonical key → items
Have I seen it before?	set	—
Count subarrays with a sum	dict	prefix sum → count
Most or least frequent	Counter + buckets or heap	item → count

9. Common bugs

1. Storing before checking in Two Sum, which pairs an element with itself.
2. Using a list as a key (`TypeError: unhashable type`). Use a tuple.
3. `d[k] += 1` on a missing key raises `KeyError` . Use `.get(k, 0)` or `defaultdict(int)` .
4. Changing a dict while looping over it. Loop over `list(d)` instead.
5. Assuming a dict is sorted. It keeps insertion order, not key order.

10. Edge cases

- Empty input, and a single element
- Duplicates, e.g. `[3, 3]` with target 6
- Negative numbers and zero
- Unicode or uppercase when the problem says "lowercase". Ask first.

11. Complexity

Operation	Average	Worst
<code>x in d</code> , <code>d[x]</code> , <code>d[x] = v</code> , <code>del d[x]</code>	$O(1)$	$O(n)$
Build from n items	$O(n)$	$O(n^2)$
Space	$O(n)$	$O(n)$

12. Interview prep

- **Purpose:** turn repeated searching into one lookup, in exchange for memory.
- **Control flow:** one pass. At each element, query the map for what you need, then update the map with the current element.
- **Trade-offs:** $O(n)$ memory. If memory is tight and the input can be sorted, sorting plus two pointers gives $O(n \log n)$ time and $O(1)$ extra space (see `twoPointers/theory.md`).
- **What a strong candidate says:** "The brute force checks all pairs, $O(n^2)$. The inner loop only answers 'does target minus x exist?', which a hash map answers in $O(1)$. So one pass with a value-to-index map gives $O(n)$ time and $O(n)$ space. I check before inserting so an element can't pair with itself."

- **Likely follow-ups:** "What if the input is sorted?" (two pointers, $O(1)$ space). "What if it doesn't fit in memory?" (split the data into hash buckets on disk, or sort externally). "Return all pairs, not just one?" (count occurrences and handle duplicates).
-

Two pointers, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved:
- Mistake and correction:

One word: Squeeze. **One sentence:** Two indices move through the array under a rule that lets you discard one element per step for sure, turning an $O(n^2)$ pair search into $O(n)$.

Executive summary

- **Opposite ends:** `i` starts left and `j` starts right, and they move toward each other. Used for sorted pair sums, palindromes and Container With Most Water.
 - **Same direction (read/write):** a fast reader and a slow writer. Used for removing duplicates, moving zeroes and in-place filtering.
 - **Fast/slow on linked lists:** cycle detection and finding the middle. See `LinkedList/theory.md`.
 - **Why it works:** every move must be provably safe. You only drop an element when no answer can use it. That argument is what the interviewer wants to hear.
 - **Cost:** $O(n)$ time and $O(1)$ space, plus $O(n \log n)$ if you have to sort first.
-

1. Opposite ends: Two Sum II, sorted input (LC 167)

```
def twoSumSorted(numbers, target):
    i, j = 0, len(numbers) - 1
    while i < j:
        s = numbers[i] + numbers[j]
        if s == target:
            return [i + 1, j + 1]    # LC 167 is 1-indexed
        if s < target:
            i += 1                    # need a bigger sum, and numbers[i] is
        else:
            j -= 1                    # need a smaller sum, and numbers[j] is
    return []
```

Trace `[2, 7, 11, 15]`, target 9: (0,3: 17 > 9, j = 2), (0,2: 13 > 9, j = 1), (0,1: 9, return [1, 2]).

Invariant: if an answer exists, it uses indices inside `[i, j]`. **Why moving `i` is safe:** if `numbers[i] + numbers[j] < target`, then `numbers[i]` plus any element left of `j` is even smaller, because the array is sorted. So `numbers[i]` can't be in any answer and can be dropped.

2. Opposite ends: Valid Palindrome (LC 125)

```
def isPalindrome(s):
    i, j = 0, len(s) - 1
    while i < j:
        if not s[i].isalnum():
            i += 1
        elif not s[j].isalnum():
            j -= 1
        elif s[i].lower() != s[j].lower():
            return False
        else:
            i += 1
            j -= 1
    return True
```

- $O(n)$ time and $O(1)$ space. The one-liner `t = [c.lower() for c in s if c.isalnum()]; return t == t[::-1]` is $O(n)$ space.
- **Follow-up, LC 680 "delete at most one character":** at the first mismatch, try skipping `i` or skipping `j` and check whether the rest is a palindrome.

3. Opposite ends: Container With Most Water (LC 11)

```
def maxArea(height):
    i, j = 0, len(height) - 1
    best = 0
    while i < j:
        best = max(best, (j - i) * min(height[i], height[j]))
        if height[i] < height[j]:
            i += 1
        else:
            j -= 1
    return best
```

Why moving the shorter line is safe: say `height[i]` is the shorter one. Any other container using line `i` has a smaller width, and its height is still at most `height[i]`. So none of them can beat the area just measured, and line `i` can be dropped.

4. Fix one element, two-pointer the rest: 3Sum (LC 15)

```
def threeSum(nums):
    nums.sort()
    res = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue # skip a repeated first value
        if nums[i] > 0:
            break # everything after is positive too
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0:
                l += 1
            elif s > 0:
                r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                l += 1
                r -= 1
                while l < r and nums[l] == nums[l - 1]:
                    l += 1 # skip a repeated second value
    return res
```

- $O(n^2)$ time, and $O(1)$ extra space apart from the output.

- **Skipping duplicates is the whole difficulty.** Skip at the outer level (`nums[i] == nums[i-1]`) and at the inner level after a match.

5. Same direction: read and write pointers

Remove Duplicates from Sorted Array (LC 26):

```
def removeDuplicates(nums):
    w = 1                                # nums[0:w] holds the unique values so far
    for r in range(1, len(nums)):
        if nums[r] != nums[w - 1]:
            nums[w] = nums[r]
            w += 1
    return w
```

Move Zeroes (LC 283):

```
def moveZeroes(nums):
    w = 0                                # nums[0:w] holds the non-zeros in the array
    for r in range(len(nums)):
        if nums[r] != 0:
            nums[w], nums[r] = nums[r], nums[w]
            w += 1
```

Invariant: everything before `w` is final. `r` only looks ahead.

6. Decision table

Signal	Variant
Sorted array, find a pair or triple with a sum	Opposite ends (fix one element for 3Sum)
Symmetry: palindrome, reversing in place	Opposite ends
Maximize something set by the two ends (area, width)	Opposite ends, move the weaker side
In-place filter, dedupe or compact	Same direction, read and write
Cycle, middle, k-th from the end in a list	Fast and slow (see <code>LinkedList/</code>)
Unsorted, and you need the original indices	Hash map instead (see <code>hashmap/</code>)

7. Common bugs

1. `while i <= j` when a pair must use two different elements. Use `i < j`.
2. Forgetting to move a pointer after a match, which loops forever.
3. 3Sum duplicates: skipping before the first use (`nums[i] == nums[i+1]`) loses valid triples. Compare with the previous element instead.
4. Two pointers on unsorted data for a sum problem. The discard argument needs sorted input.
5. Returning 0-based indices when the problem is 1-based (LC 167).

8. Edge cases

- Length 0, 1 and 2
- All duplicates, e.g. `[0, 0, 0, 0]` for 3Sum should return one triple
- Negative numbers
- Strings with only punctuation for LC 125 should return True

9. Interview prep

- **Purpose:** examine pairs without checking all of them, by using order to discard candidates.
- **Control flow:** start the pointers, compare, move exactly the pointer that the discard rule allows, and stop when they meet.
- **Trade-offs:** needs sorted input or a monotonic structure. Sorting loses the original indices, so keep `(value, index)` pairs if you need them. Compared with a hash map, it uses less memory ($O(1)$) but takes more time if you have to sort.
- **What a strong candidate says:** "The array is sorted, so if the sum is too small, the left element can't work with anything, and I can drop it. Every step drops one element, so it's $O(n)$ with $O(1)$ space."
- **Likely follow-ups:** 3Sum Closest (LC 16), 4Sum (LC 18, fix two elements), Trapping Rain Water (LC 42, move the side with the lower max), "what if there are duplicates?"

Sliding window, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved: 121
- Mistake and correction:

One word: Window. **One sentence:** Keep a contiguous range `[l, r]` that you grow on the right and shrink on the left, updating its summary as you go, so every subarray question is answered in one pass.

Executive summary

- **Use it for:** contiguous subarrays or substrings: the longest, shortest or count that satisfies a condition.
- **Two kinds:** fixed size k (slide by one), and variable size (grow `r`, shrink `l` while the window is invalid).
- **State:** a running summary of the window, such as a sum, a character count or the number of distinct characters. Update it in $O(1)$ when a character enters or leaves.
- **Why $O(n)$:** `r` moves n times, and `l` moves at most n times in total. The inner `while` doesn't make it $O(n^2)$.
- **When it fails:** the condition must stay monotonic. Shrinking must never turn a valid window invalid in the wrong direction. Sums with negative numbers break this, so use a prefix sum plus a hash map instead.

1. The variable-window template

```
def longest_valid(s):
    l = 0
    state = {}                                # whatever describes window s[l..r]
    best = 0
    for r in range(len(s)):
        # 1. add s[r] to state
        # 2. while the window is invalid: remove s[l] from state, l += 1
        # 3. the window is valid now: best = max(best, r - l + 1)
        pass
    return best
```

For **shortest** questions, flip it: while the window **is valid**, record the answer, then shrink.

2. Longest Substring Without Repeating Characters (LC 3)

```
def lengthOfLongestSubstring(s):
    last = {}                                # char -> last index seen
    l = 0
    best = 0
    for r, ch in enumerate(s):
        if ch in last and last[ch] >= l:
            l = last[ch] + 1                # jump past the previous copy
        last[ch] = r
        best = max(best, r - l + 1)
    return best
```

Trace "abcabcbb":

r	ch	last[ch] before	l after	window	best
0	a	—	0	a	1
1	b	—	0	ab	2
2	c	—	0	abc	3
3	a	0	1	bca	3
4	b	1	2	cab	3
5	c	2	3	abc	3
6	b	4	5	cb	3
7	b	6	7	b	3

Invariant: `s[l..r]` has no repeated character. **Why** `last[ch] >= l`: an old copy that's already left of the window doesn't matter. Without this check, `"abba"` moves `l` backwards and returns the wrong answer.

3. Shortest window: Minimum Size Subarray Sum (LC 209)

```
def minSubArrayLen(target, nums):
    l = 0
    s = 0
    best = float('inf')
    for r, x in enumerate(nums):
        s += x
        while s >= target:
            # valid, so record it, then try to shrink
            best = min(best, r - l + 1)
            s -= nums[l]
            l += 1
    return 0 if best == float('inf') else best
```

This only works because all numbers are positive: adding makes the sum bigger and removing makes it smaller.

4. Longest Repeating Character Replacement (LC 424)

A window is valid if `window length - count of its most frequent char <= k`, meaning at most `k` characters need replacing.

```
def characterReplacement(s, k):
    count = {}
    l = 0
    maxf = 0
    best = 0
    for r, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        maxf = max(maxf, count[ch])
        while (r - l + 1) - maxf > k:
            count[s[l]] -= 1
            l += 1
        best = max(best, r - l + 1)
    return best
```

Why `maxf` never needs to go down: a stale, too-high `maxf` can only stop the window from growing past its best size so far. `best` only increases when a real, higher frequency appears. This is a classic follow-up question, so be ready to explain it.

5. Fixed-size window

Maximum Average Subarray I (LC 643), or any "every window of size k":

```
def findMaxAverage(nums, k):
    s = sum(nums[:k])
    best = s
    for r in range(k, len(nums)):
        s += nums[r] - nums[r - k]      # one element enters, one leaves
        best = max(best, s)
    return best / k
```

Permutation in String (LC 567) is a fixed window of `len(s1)` that compares 26-letter counts.

6. Best Time to Buy and Sell Stock (LC 121) as a window

```
def maxProfit(prices):
    lowest = float('inf')
    best = 0
    for p in prices:
        lowest = min(lowest, p)          # the best buy day so far is the window
        best = max(best, p - lowest)
    return best
```

7. Decision table

Question	Window	Loop shape
Longest with at most / no more than ...	variable	shrink while invalid, then record
Shortest with at least ...	variable	record while valid, then shrink
Every window of size k	fixed	add the right element, remove element r-k
Count subarrays with exactly K ...	variable twice	atMost(K) – atMost(K-1)
Subarray sum = k with negative numbers	not a window	prefix sum + hash map

8. Common bugs

1. Recording the answer before the window is fixed up (longest) or after it's been shrunk (shortest).
2. Forgetting to remove `s[l]` from the state before moving `l`.
3. Moving `l` backwards (see the "abba" case in LC 3).
4. Using a window on sums with negative numbers.
5. Off-by-one in the length: it's `r - l + 1`.

9. Edge cases

- Empty string or array
- $k = 0$, and $k \geq n$

- Everything the same character, or all characters different
- No valid window at all (LC 209 should return 0)

10. Interview prep

- **Purpose:** answer questions about all contiguous ranges in $O(n)$ instead of $O(n^2)$.
 - **Control flow:** a `for` loop grows `r`, an inner `while` shrinks `l` until the condition holds again, then you record the answer.
 - **Complexity:** $O(n)$ time, because each index enters and leaves once. Space is $O(\text{size of the alphabet})$ or $O(k)$.
 - **What a strong candidate says:** "Each index enters the window once and leaves once, so even with the inner while loop, it's $O(n)$. The window is valid when..., and shrinking only ever makes it more valid, which is why the greedy shrink is correct."
 - **Likely follow-ups:** Minimum Window Substring (LC 76, hard), Sliding Window Maximum (LC 239, uses a monotonic deque), "at most K distinct characters".
-

Prefix sum, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved: 238
- Mistake and correction:

One word: Accumulate. **One sentence:** Precompute running totals once, so the sum of any range becomes one subtraction instead of a loop.

Executive summary

- **Definition:** $P[0] = 0$ and $P[i+1] = P[i] + \text{nums}[i]$. So $P[i]$ is the sum of the first i elements.
- **Range sum:** $\text{sum}(\text{nums}[l..r]) = P[r+1] - P[l]$, in $O(1)$ after an $O(n)$ build.
- **The big trick:** "subarray sum equals k " becomes "find an earlier prefix equal to $\text{current} - k$ ", which a hash map answers. It works with negative numbers, where a sliding window fails.
- **Related ideas:** prefix products (LC 238), 2-D prefix sums, difference arrays (for range updates), and prefix XOR.

1. Build and query

```
def build_prefix(nums):
    P = [0] * (len(nums) + 1)
    for i, x in enumerate(nums):
        P[i + 1] = P[i] + x
    return P

def range_sum(P, l, r):
    # inclusive l..r
    return P[r + 1] - P[l]
```

Example: `nums = [3, 1, 4, 1, 5]` gives `P = [0, 3, 4, 8, 9, 14]`. Then `sum(nums[1..3]) = P[4] - P[1] = 9 - 3 = 6`, which checks out: $1 + 4 + 1$. **Why the extra leading 0:** it makes ranges that start at index 0 work with no special case. `sum(nums[0..r]) = P[r+1] - P[0]`. `itertools.accumulate(nums, initial=0)` builds the same list.

2. Find Pivot Index (LC 724)

The pivot is where the left sum equals the right sum. The right sum is `total - left - nums[i]`.

```
def pivotIndex(nums):
    total = sum(nums)
    left = 0
    for i, x in enumerate(nums):
        if left == total - left - x:
            return i
        left += x
    return -1
```

Trace `[1, 7, 3, 6, 5, 6]`, total 28:

i	x	left	right = 28 - left - x	equal?
0	1	0	27	no
1	7	1	20	no
2	3	8	17	no
3	6	11	11	yes, return 3

3. Subarray Sum Equals K (LC 560): prefix sum plus a hash map

A subarray `nums[j..i]` sums to `k` exactly when `P[i+1] - P[j] = k`, which means `P[j] = P[i+1] - k`. So for each position, count how many earlier prefixes equal `current - k`.

```
def subarraySum(nums, k):
    count = {0: 1}  # the empty prefix, for subarrays starting at index 0
    s = 0
    res = 0
    for x in nums:
        s += x
        res += count.get(s - k, 0)
        count[s] = count.get(s, 0) + 1
    return res
```

Trace `[1, 1, 1]`, `k = 2`:

x	s	need s-k	count before	res
1	1	-1	{0:1}	0
1	2	0	{0:1, 1:1}	1
1	3	1	{0:1, 1:1, 2:1}	2

- **Why `{0: 1}`:** without it, a subarray that starts at index 0 is never counted.
- **Why look up before storing:** otherwise, with `k = 0`, a prefix would match itself and count an empty subarray.
- **Why not a sliding window:** with negative numbers, growing the window doesn't always make the sum bigger, so the shrink rule breaks.

4. Contiguous Array (LC 525): turning a count into a sum

Longest subarray with equally many 0s and 1s. Treat 0 as `-1`, so the question becomes "longest subarray with sum 0". Store the **first** index each prefix sum was seen.

```
def findMaxLength(nums):
    first = {0: -1}
    s = 0
    best = 0
    for i, x in enumerate(nums):
        s += 1 if x == 1 else -1
        if s in first:
            best = max(best, i - first[s])
        else:
            first[s] = i          # keep the earliest index, which gives the longest subarray
    return best
```

5. Prefix and suffix products: Product of Array Except Self (LC 238)

```
def productExceptSelf(nums):
    n = len(nums)
    res = [1] * n
    pre = 1
    for i in range(n):          # res[i] = product of everything left of i
        res[i] = pre
        pre *= nums[i]
    suf = 1
    for i in range(n - 1, -1, -1): # multiply in the product of everything right of i
        res[i] *= suf
        suf *= nums[i]
    return res
```

$O(n)$ time, $O(1)$ extra space apart from the output, and no division, so zeros are handled.

6. 2-D prefix sum (Range Sum Query 2D, LC 304)

$P[r+1][c+1]$ is the sum of the rectangle from $(0,0)$ to (r,c) .

- Build: $P[r+1][c+1] = \text{grid}[r][c] + P[r][c+1] + P[r+1][c] - P[r][c]$
- Query the rectangle $(r1,c1)$ to $(r2,c2)$: $P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]$

This is inclusion and exclusion: subtract the two strips, then add back the corner that was subtracted twice.

7. Difference array (range updates)

To add `v` to every element of `nums[l..r]` many times, record the updates, then build once:

```
diff = [0] * (n + 1)
diff[l] += v
diff[r + 1] -= v
# afterwards: running sum of diff gives the total added at each index
```

Each update is $O(1)$, and one $O(n)$ pass applies them all. Used in Car Pooling (LC 1094) and Corporate Flight Bookings (LC 1109).

8. Decision table

Question	Tool
Many range-sum queries on fixed data	Prefix array
Count or length of subarrays with sum = k (negatives allowed)	Prefix sum + hash map
Longest subarray with a balanced count	Map each value to ± 1 , then prefix sum + first-seen map
Product of everything except self	Prefix and suffix products
Many range updates, then read	Difference array
Sum of a rectangle	2-D prefix sum
Data changes between queries	Fenwick tree or segment tree (outside this roadmap)

9. Common bugs

1. Forgetting the leading 0, then special-casing `l = 0` and getting it wrong.
2. Off-by-one: the range `[l, r]` is `P[r+1] - P[l]`, not `P[r] - P[l]`.
3. LC 560: storing the current prefix before looking it up.
4. LC 525: overwriting the first-seen index, which shrinks the answer.
5. Using a sliding window when the input has negative numbers.

10. Edge cases

- Empty array, and a single element
- $k = 0$, and all zeros (LC 560 with `[0, 0, 0]` and $k = 0$ should give 6)
- Negative numbers
- A pivot at index 0 or $n-1$

11. Interview prep

- **Purpose:** replace repeated range loops with $O(1)$ subtraction after an $O(n)$ setup.
 - **Control flow:** one pass to build or keep a running sum. For counting questions, look up `current - k` in a map, then record `current`.
 - **Complexity:** $O(n)$ time. $O(n)$ space for the array or map, or $O(1)$ for a single running total.
 - **What a strong candidate says:** "A subarray sum is the difference of two prefix sums, so 'sum equals k ' means 'an earlier prefix equals the current one minus k '. A hash map of prefix counts makes it $O(n)$, and it works with negative numbers, where a sliding window wouldn't."
 - **Likely follow-ups:** Continuous Subarray Sum (LC 523, prefix mod k), Subarray Sums Divisible by K (LC 974), "what if the array changes?" (Fenwick tree).
-

Stack, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved:
- Mistake and correction:

One word: LIFO. **One sentence:** A stack remembers unfinished items in the order they must be finished, most recent first, which fits anything nested or anything waiting for its next greater or smaller value.

Executive summary

- **Operations:** push, pop and peek are all $O(1)$. In Python, a list: `append`, `pop()`, `st[-1]`.
- **Shape 1, matching:** brackets, tags, undo. Push when something opens, pop when it closes.
- **Shape 2, evaluating:** Reverse Polish Notation (RPN), calculators. Push operands, pop two when you see an operator.
- **Shape 3, extra information:** Min Stack stores each value with the minimum at the time it was pushed.
- **Shape 4, monotonic stack:** keep the stack increasing or decreasing to find the "next greater" or "previous smaller" element for every index in $O(n)$ total.
- **Also:** DFS is a stack, and recursion uses the call stack.

1. Matching: Valid Parentheses (LC 20)

```
def isValid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    st = []
    for ch in s:
        if ch in pairs:
            # a closing bracket
            if not st or st[-1] != pairs[ch]:
                return False
            st.pop()
        else:
            # an opening bracket
            st.append(ch)
    return not st
    # anything left open means not valid
```

Trace "`([)]`": push `(`, push `[`, then `)` arrives but the top is `[`, which doesn't match, so return False. Trace "`{()}`": push `(`, push `{`, `}` matches and pops, `)` matches and pops, and the stack is empty, so return True. **Invariant:** the stack holds exactly the brackets that are open and not yet closed, most recent on top.

2. Extra information: Min Stack (LC 155)

```
class MinStack:
    def __init__(self):
        self.st = []
        # (value, min at or below this value)

    def push(self, val):
        m = min(val, self.st[-1][1]) if self.st else val
        self.st.append((val, m))

    def pop(self):
        self.st.pop()

    def top(self):
        return self.st[-1][0]

    def getMin(self):
        return self.st[-1][1]
```

Every operation is $O(1)$. Each item stores the minimum of everything below it, so popping automatically brings back the previous minimum.

3. Evaluating: Evaluate Reverse Polish Notation (LC 150)

```
def evalRPN(tokens):
    st = []
    for t in tokens:
        if t in {'+', '-', '*', '/'}:
            b = st.pop()          # the second operand comes
            a = st.pop()
            if t == '+': st.append(a + b)
            elif t == '-': st.append(a - b)
            elif t == '*': st.append(a * b)
            else: st.append(int(a / b))  # truncate toward zero, not
        else:
            st.append(int(t))
    return st[0]
```

Bug to avoid: `a // b` rounds down ($-7 // 2 = -4$), but the problem wants rounding toward zero (-3). Use `int(a / b)`.

4. Monotonic stack: Daily Temperatures (LC 739)

Question: for each day, how many days until a warmer one? **Idea:** keep a stack of indices still waiting for a warmer day. Their temperatures are decreasing from bottom to top. A new temperature pops every waiting day that's colder, and those days have found their answer.

```
def dailyTemperatures(t):
    res = [0] * len(t)
    st = []  # indices, temperatures dec
    for i, x in enumerate(t):
        while st and t[st[-1]] < x:
            j = st.pop()
            res[j] = i - j
        st.append(i)
    return res
```

Trace `[73, 74, 75, 71, 69, 72, 76, 73]`:

i	temp	pops (and their answers)	stack after (temps)
0	73	—	[73]
1	74	day 0 → 1	[74]
2	75	day 1 → 1	[75]
3	71	—	[75, 71]
4	69	—	[75, 71, 69]
5	72	day 4 → 1, day 3 → 2	[75, 72]
6	76	day 5 → 1, day 2 → 4	[76]
7	73	—	[76, 73]

Result: `[1, 1, 4, 2, 1, 1, 0, 0]`. Days left on the stack never find a warmer day and keep 0.

Why $O(n)$ with a nested loop: each index is pushed once and popped at most once, so there are $2n$ operations in total.

5. Next Greater Element I (LC 496)

Same idea, with a map from each value to its next greater value:

```
def nextGreaterElement(nums1, nums2):
    nxt = {}
    st = []
    for x in nums2:
        while st and st[-1] < x:
            nxt[st.pop()] = x
        st.append(x)
    return [nxt.get(x, -1) for x in nums1]
```

6. Which monotonic stack to use

You want, for each element...	Loop direction	Stack keeps	Pop while
Next greater to the right	left → right	decreasing	top < current
Next smaller to the right	left → right	increasing	top > current
Previous greater to the left	left → right	decreasing	top ≤ current, then the top is the answer
Previous smaller to the left	left → right	increasing	top ≥ current, then the top is the answer

Harder problems built on this: Largest Rectangle in Histogram (LC 84, previous and next smaller), Trapping Rain Water (LC 42), Car Fleet (LC 853), Online Stock Span (LC 901).

7. Decision table

Signal	Shape
Nested structure, open and close pairs	Matching
Postfix expression, "evaluate"	Evaluating
O(1) min or max while pushing and popping	Store extra info per item
"Next / previous greater / smaller", "days until", "span"	Monotonic stack
First in, first out (queue order) needed	<code>collections.deque</code> , not a stack

8. Common bugs

1. Popping from an empty stack. Always check `if st` first.
2. Forgetting the final `return not st`, so `"(("` passes.
3. RPN: getting the operand order wrong (`b` is popped first) or using floor division.
4. Monotonic stack: storing values when you need distances. Store indices, since you can always look up the value from an index.
5. Using `list.pop(0)` as a queue, which is O(n). Use `deque.popleft()`.

9. Edge cases

- Empty string, and a single character
- Only closing brackets, e.g. `)")"`
- Strictly increasing or decreasing temperatures
- Equal values: decide whether "greater" means strictly greater

10. Interview prep

- **Purpose:** handle nesting and "what's the nearest bigger or smaller one?" in one pass.
- **Control flow:** scan once. When the current item resolves the top of the stack, pop and record. Then push the current item.
- **Complexity:** $O(n)$ time, because each item is pushed and popped at most once. $O(n)$ space in the worst case.
- **What a strong candidate says:** "I keep a stack of indices still waiting for a greater element, so it stays decreasing. Each new element pops everything smaller, and those indices have their answer. Each index is pushed and popped once, so it's $O(n)$ even with the inner loop."
- **Likely follow-ups:** Largest Rectangle in Histogram (LC 84), Basic Calculator (LC 224), Decode String (LC 394), a queue built from two stacks (LC 232).

Intervals, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
 - Invariant:
 - Template/pseudocode from memory:
 - Complexity:
 - Solved/re-solved:
 - Mistake and correction:
-

One word: Overlap. **One sentence:** Sort intervals by start (or by end), then sweep once, comparing each interval only with the last one you kept.

Executive summary

- **Overlap test:** $[a, b]$ and $[c, d]$ overlap when $a \leq d$ and $c \leq b$. Whether touching ends ($b == c$) count as overlapping depends on the problem, so read it carefully.
 - **Merging:** sort by start. Each interval either extends the last merged interval or starts a new one.
 - **Keeping as many as possible without overlap:** sort by **end**, and greedily keep whichever interval finishes earliest.
 - **Counting how many overlap at once (meeting rooms):** sort the starts and ends separately, or use a min-heap of end times.
 - **Cost:** $O(n \log n)$ for the sort, then an $O(n)$ sweep.
-

1. Merge Intervals (LC 56)

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    res = []
    for s, e in intervals:
        if res and s <= res[-1][1]:
            res[-1][1] = max(res[-1][1], e)
        else:
            res.append([s, e])
    return res
```

Trace `[[1, 3], [2, 6], [8, 10], [15, 18]]` (already sorted):

interval	last merged	overlaps?	res
[1,3]	—	—	[[1,3]]
[2,6]	[1,3]	$2 \leq 3$, yes	[[1,6]]
[8,10]	[1,6]	$8 \leq 6$? no	[[1,6],[8,10]]
[15,18]	[8,10]	no	[[1,6],[8,10],[15,18]]

Invariant: after sorting, an interval can only overlap the **last** merged interval. Earlier ones end before the last one starts. **Why** `max`: `[1, 10]` followed by `[2, 3]` must stay `[1, 10]`, not shrink to `[1, 3]`.

2. Insert Interval (LC 57): already sorted, $O(n)$

Three phases: intervals entirely before the new one, intervals overlapping it (merge them in), and intervals entirely after.

```
def insert(intervals, new):
    res = []
    i, n = 0, len(intervals)
    while i < n and intervals[i][1] < new[0]:          # ends before new starts
        res.append(intervals[i])
        i += 1
    while i < n and intervals[i][0] <= new[1]:        # overlaps new
        new = [min(new[0], intervals[i][0]), max(new[1], intervals[i][1])]
        i += 1
    res.append(new)
    res.extend(intervals[i:])                        # starts after new ends
    return res
```

Trace `[[1,2],[3,5],[6,7],[8,10],[12,16]]` with new `[4,8]`: `[1,2]` goes before. `[3,5]`, `[6,7]` and `[8,10]` merge into `[3,10]`. `[12,16]` goes after. Result: `[[1,2],[3,10],[12,16]]`.

3. Non-overlapping Intervals (LC 435): sort by end

Remove as few intervals as possible so that none overlap. That's the same as keeping as many as possible.

```
def eraseOverlapIntervals(intervals):
    intervals.sort(key=lambda x: x[1])
    end = float('-inf')
    keep = 0
    for s, e in intervals:
        if s >= end:                # touching is allowed in this problem
            keep += 1
            end = e
    return len(intervals) - keep
```

Why sort by end (the exchange argument): take any best solution. Its first interval can be swapped for the interval that ends earliest without causing any overlap, because ending earlier only leaves more room. Repeat that for each step, and the greedy choice is optimal. **Why not sort by start:** one long interval that starts early would block many short ones.

4. Minimum Number of Arrows to Burst Balloons (LC 452)

The same greedy idea: sort by end, and shoot at the end of the first balloon still standing.

```
def findMinArrowShots(points):
    points.sort(key=lambda x: x[1])
    arrows = 0
    end = float('-inf')
    for s, e in points:
        if s > end:
            arrows += 1
            end = e
    return arrows
```

Compare with LC 435: the only difference is `>` versus `>=`, depending on whether touching counts. Read every problem's definition of overlap.

5. Meeting Rooms II (LC 253): the most rooms needed at once

```
def minMeetingRooms(intervals):
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = 0
    j = 0
    for s in starts:
        if s < ends[j]:
            rooms += 1
        else:
            j += 1
    return rooms
```

Trace `[[0, 30], [5, 10], [15, 20]]`: starts `[0, 5, 15]`, ends `[10, 20, 30]`. $0 < 10 \rightarrow 1$ room. $5 < 10 \rightarrow 2$ rooms. $15 \geq 10 \rightarrow$ reuse ($j = 1$). ****Answer: 2.**** ****Heap version:**** sort by start, and keep a min-heap of end times. Pop the earliest end if it's $\leq$ the current start, then push the current end. The heap's size at the end is the answer. See `heap/theory.md`. Meeting Rooms I (LC 252, "can one person attend all?"): sort by start and check `intervals[i][0] >= intervals[i-1][1]` for every i .

6. Decision table

Question	Sort by	Sweep rule
Merge overlapping	start	extend the last one or start a new one
Insert into an already sorted list	(already sorted)	before / overlap / after phases
Keep the most, or remove the fewest	end	keep if $\text{start} \geq$ the last kept end
Fewest points that hit every interval	end	new point if $\text{start} >$ the last point
Most intervals overlapping at once	starts and ends separately, or a heap	two pointers or a heap of ends
Can one person attend everything?	start	any overlap means no

7. Common bugs

1. Forgetting to sort, or sorting by the wrong end.
2. Replacing the end instead of taking `max(old_end, new_end)`.
3. Mixing up `<` and `<=` for touching intervals. Check an example like `[1, 2], [2, 3]` against the problem statement.
4. Changing the input list when the caller still needs it. Say so, or copy it first.
5. Changing a tuple in place. Store lists, or build new tuples.

8. Edge cases

- Empty list, or one interval
- One interval fully inside another
- Touching endpoints
- All intervals identical
- Intervals given out of order

9. Interview prep

- **Purpose:** make overlap questions local. After sorting, you only ever compare with the previous interval.
 - **Control flow:** sort, then one pass keeping a single "current" interval or end value.
 - **Complexity:** $O(n \log n)$ time for the sort. $O(n)$ space for the output, or $O(1)$ extra.
 - **What a strong candidate says:** "Once sorted by start, an interval can only overlap the last merged one, so a single pass works. For keeping the most intervals, I sort by end, and the exchange argument proves it's optimal: the earliest-ending interval leaves the most room."
 - **Likely follow-ups:** Employee Free Time (LC 759), Interval List Intersections (LC 986, two pointers), Minimum Interval to Include Each Query (LC 1851, sort plus a heap).
-

Linked list, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved:
- Mistake and correction:

One word: Rewiring. **One sentence:** Linked-list problems come down to moving a few pointers in the right order, and a dummy head, the fast/slow technique and "save `next` before you overwrite it" handle almost all of them.

Executive summary

- **The structure:** each node has a `val` and a `next`. Access is $O(n)$, but inserting or deleting at a known node is $O(1)$.
- **Four tools:**
 1. **Save next first:** `next = cur.next` before changing `cur.next`.
 2. **Dummy head:** a fake node before the real head, so deleting or inserting at the head needs no special case.
 3. **Fast and slow pointers:** fast moves 2 steps, slow moves 1. Used for the middle, cycle detection, and the start of a cycle.
 4. **Gap pointers:** move one pointer n steps ahead, then move both together. Used for "n-th from the end".
- **Draw it.** Most bugs are a pointer changed in the wrong order. Sketch the boxes and arrows for 3 nodes before writing code.

0. Helpers for local testing (LeetCode provides `ListNode`)

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def build(vals):
    dummy = ListNode()
    cur = dummy
    for v in vals:
        cur.next = ListNode(v)
        cur = cur.next
    return dummy.next

def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out
```

1. Reverse Linked List (LC 206)

```
def reverseList(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next      # 1. save
        cur.next = prev     # 2. reverse this node's arrow
        prev = cur          # 3. move prev forward
        cur = nxt           # 4. move cur forward
    return prev
```

Trace `1 → 2 → 3`:

step	prev	cur	reversed part so far
start	None	1	—
1	1	2	1 → None
2	2	3	2 → 1 → None
3	3	None	3 → 2 → 1 → None

Invariant: `prev` is the head of the reversed part, and `cur` is the head of the part not yet reversed. Recursive version: `rest = reverseList(head.next); head.next.next = head; head.next = None; return rest`, with a base case of an empty list or one node. That's $O(n)$ stack space.

2. Merge Two Sorted Lists (LC 21), with a dummy head

```
def mergeTwoLists(a, b):
    dummy = ListNode()
    tail = dummy
    while a and b:
        if a.val <= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    tail.next = a or b          # attach whatever is left
    return dummy.next
```

The dummy means you never have to ask "is this the first node?".

3. Linked List Cycle (LC 141): Floyd's algorithm

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False
```

Why they must meet: once both are inside the cycle, fast gains 1 node on slow each step. The gap shrinks by 1 every step, so it reaches 0. $O(n)$ time, $O(1)$ space. A `set` of visited nodes also works, but needs $O(n)$ space. **Where the cycle starts (LC 142):** after they meet, move one pointer back to `head`, then move both one step at a time. They meet at the cycle's start. The distance from the head to the start equals the distance from the meeting point onward to the start, going around the cycle.

4. Middle of the Linked List (LC 876)

```
def middleNode(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow # with an even length, this is the second of
```

5. Remove Nth Node From End (LC 19): gap pointers

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n + 1): # put a gap of n nodes between them
        fast = fast.next
    while fast:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next # slow is just before the node to remove
    return dummy.next
```

Why a dummy: when n equals the list length, the head itself is removed, and `slow` stays on the dummy.

6. Reorder List (LC 143): combining the tools

Find the middle (fast and slow), reverse the second half (LC 206), then weave the two halves together (similar to LC 21). A common interview question because it tests all three tools at once.

7. Decision table

Question	Tool
Reverse all or part of the list	<code>prev / cur / nxt</code>
Build a new list, or delete that might remove the head	Dummy head
Cycle? Middle?	Fast and slow
k-th from the end	Gap of k
Palindrome list	Middle, reverse second half, compare
Intersection of two lists (LC 160)	Two pointers that switch lists when they reach the end

8. Common bugs

1. Losing the rest of the list by overwriting `cur.next` before saving it.
2. `fast.next.next` when `fast.next` is `None`. The loop condition must be `fast and fast.next`.
3. Returning `head` instead of `dummy.next` after the head was removed.
4. Comparing nodes with `==` when you mean the same object. Use `is`.
5. An infinite loop after rewiring (a cycle you created by accident). Set the last node's `next` to `None`.

9. Edge cases

- Empty list (`head = None`)
- One node, and two nodes
- Removing the head, and removing the tail

- Even and odd lengths for the middle

10. Interview prep

- **Purpose:** restructure data in place in $O(1)$ extra space by changing references instead of copying.
 - **Control flow:** walk with one or two pointers, save the next node, rewire, and advance.
 - **Complexity:** almost always $O(n)$ time and $O(1)$ space. Recursive solutions add $O(n)$ stack space.
 - **What a strong candidate says:** "I'll use a dummy head so removing the first node isn't a special case. I save `next` before changing any pointer. For the cycle, fast gains one step per iteration inside the loop, so they must meet, with $O(1)$ space."
 - **Likely follow-ups:** Reverse Nodes in k-Group (LC 25), Copy List with Random Pointer (LC 138), LRU Cache (LC 146, a hash map plus a doubly linked list), Add Two Numbers (LC 2).
-

Trees (DFS and BFS), in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved:
- Mistake and correction:

One word: Recursion. **One sentence:** Solve a tree problem by deciding what one node needs from its children (DFS), or by processing it one level at a time (BFS).

Executive summary

- **DFS (depth-first search):** recursion or an explicit stack. The order decides when you handle the node:
 - **preorder** (node, left, right): the node first, like copying a tree
 - **inorder** (left, node, right): gives a BST in sorted order
 - **postorder** (left, right, node): children first, used for heights and sizes
- **BFS (breadth-first search):** a queue, one level at a time. Use it for level order, right side view, and the minimum depth.
- **The recursive question:** "If I knew the answer for the left and right subtrees, how do I get the answer for this node?" Answer that, then add the base case `if not root`.
- **Two value flows:** values going **down** as parameters (bounds, path sums), and values coming **up** as return values (height, balanced or not).
- **Cost:** $O(n)$ time. $O(h)$ stack space for DFS, where h is the height. $O(\text{width})$ queue space for BFS.

0. Helpers for local testing

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def build(vals):
    """Build from a LeetCode-style level-order list, e.g. [3,9,20,None,None,15,7]
    if not vals or vals[0] is None:
        return None
    root = TreeNode(vals[0])
    q = deque([root])
    i = 1
    while q and i < len(vals):
        node = q.popleft()
        if i < len(vals) and vals[i] is not None:
            node.left = TreeNode(vals[i])
            q.append(node.left)
        i += 1
        if i < len(vals) and vals[i] is not None:
            node.right = TreeNode(vals[i])
            q.append(node.right)
        i += 1
    return root
```

1. Maximum Depth (LC 104): information flowing up

```
def maxDepth(root):
    if not root:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

Trace `[3, 9, 20, None, None, 15, 7]`: $\text{depth}(9) = 1$, $\text{depth}(15) = \text{depth}(7) = 1$, $\text{depth}(20) = 2$, $\text{depth}(3) = 1 + \max(1, 2) = 3$.

2. Traversals: Binary Tree Inorder (LC 94)

Recursive version:

```
def inorder(root):
    res = []
    def dfs(node):
        if not node:
            return
        dfs(node.left)
        res.append(node.val)
        dfs(node.right)
    dfs(root)
    return res
```

Iterative version, which interviewers often ask for:

```
def inorderTraversal(root):
    res, st, cur = [], [], root
    while cur or st:
        while cur:                # go as far left as possible
            st.append(cur)
            cur = cur.left
        cur = st.pop()            # the leftmost node not yet visited
        res.append(cur.val)
        cur = cur.right           # then its right subtree
    return res
```

For preorder or postorder, move the `append(node.val)` line before or after the recursive calls.

3. Invert Binary Tree (LC 226)

```
def invertTree(root):
    if root:
        root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

4. BFS level order (LC 102)

```
def levelOrder(root):
    if not root:
        return []
    q = deque([root])
    res = []
    while q:
        level = []
        for _ in range(len(q)):      # exactly the nodes of this level
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        res.append(level)
    return res
```

Invariant: at the start of each `while` iteration, the queue holds exactly one full level. Variations: right side view (LC 199, the last value of each level) and minimum depth (LC 111, the first leaf found by BFS).

5. Validate BST (LC 98): information flowing down

```
def isValidBST(root, lo=float('-inf'), hi=float('inf')):
    if not root:
        return True
    if not (lo < root.val < hi):
        return False
    return (isValidBST(root.left, lo, root.val) and
            isValidBST(root.right, root.val, hi))
```

The classic bug: only checking `left.val < node.val < right.val`. Tree `[5, 4, 6, None, None, 3, 7]` passes that check, but 3 is in 5's right subtree, so it's invalid. Every node must fit the bounds of **all** its ancestors, which is why the bounds are passed down. Alternative: an inorder traversal must be strictly increasing.

6. Diameter (LC 543): return one thing, track another

```
def diameterOfBinaryTree(root):
    best = 0
    def height(node):
        nonlocal best
        if not node:
            return 0
        l, r = height(node.left), height(node.right)
        best = max(best, l + r)      # the longest path passing through this node
        return 1 + max(l, r)        # what the parent needs
    height(root)
    return best
```

This "return height, update a global best" shape also solves Balanced Binary Tree (LC 110) and Binary Tree Maximum Path Sum (LC 124).

7. Lowest Common Ancestor of a BST (LC 235)

```
def lowestCommonAncestor(root, p, q):
    while root:
        if p.val < root.val and q.val < root.val:
            root = root.left
        elif p.val > root.val and q.val > root.val:
            root = root.right
        else:
            return root
    # p and q split here, or one of them is the root
```

8. Decision table

Question	Approach
Height, size, balanced, diameter	Postorder DFS: the answer comes up from the children
BST valid, path sum from the root	Preorder DFS: pass bounds or a running sum down
Sorted output from a BST, k-th smallest	Inorder
Level by level, right view, nearest leaf	BFS with a queue
Very deep tree (risk of recursion limit)	Iterative with a stack

9. Common bugs

1. A missing `if not root` base case, causing an `AttributeError` on `None`.
2. BST validation that only compares with the direct children.
3. BFS without the `for _ in range(len(q))` loop, which mixes levels together.
4. Python's recursion limit (about 1000) on a skewed tree. Use an iterative version or `sys.setrecursionlimit`.
5. Collecting paths with `path` instead of `path[:]` (same bug as in backtracking).

10. Edge cases

- Empty tree, and a single node
- A completely skewed tree (a linked list in disguise, height n)
- Duplicate values (read the problem's BST rule: `<` or `<=`)
- Negative values in path-sum problems

11. Interview prep

- **Purpose:** process hierarchical data by breaking it into subtrees.
- **Control flow:** base case, recurse into the children, combine the results, return. For BFS: queue, level loop, add the children.
- **Complexity:** $O(n)$ time. Space is $O(h)$ for DFS: $O(\log n)$ if balanced, $O(n)$ if skewed. $O(w)$ for BFS, where w is the widest level.
- **What a strong candidate says:** "I'll define what each call returns, here the subtree's height. The answer at a node combines its children's heights, and the diameter through a node is left plus right. Each node is visited once, so it's $O(n)$ time and $O(h)$ stack space."
- **Likely follow-ups:** Serialize/Deserialize (LC 297), build a tree from preorder and inorder (LC 105), Kth Smallest in a BST (LC 230), LCA of a general binary tree (LC 236).

Heap / priority queue, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved:
- Mistake and correction:

One word: Priority. **One sentence:** A heap always hands you the smallest (or largest) item in $O(\log n)$, which is exactly what "top k", "k-th largest", "merge sorted streams" and "the next event to happen" need.

Executive summary

- **Python's** `heapq` **is a min-heap** stored in a plain list. `h[0]` is always the smallest.
- **Costs:** `heappush` and `heappop` are $O(\log n)$, peeking at `h[0]` is $O(1)$, and `heapify` is $O(n)$.
- **Max-heap:** push `-x` and negate again when you pop.
- **Top k largest:** keep a **min-heap** of size k. Its root is the k-th largest. $O(n \log k)$ time, $O(k)$ space.
- **Ties and objects that can't be compared:** push tuples like `(priority, counter, item)`.

1. How it works (just enough)

- It's a complete binary tree stored in an array. The children of index i are at $2i+1$ and $2i+2$, and its parent is at $(i-1)//2$.
- **Heap property:** every parent is $\leq$ its children. So the minimum is at the root. The rest isn't sorted.
- **Push:** add at the end, then swap upward while it's smaller than its parent. $O(\log n)$.
- **Pop:** move the last element to the root, then swap downward with its smaller child. $O(\log n)$.
- **heapify is $O(n)$, not $O(n \log n)$:** most nodes are near the bottom and only move a short distance.

```
import heapq
h = [5, 1, 8, 3]
heapq.heapify(h)           # h[0] == 1
heapq.heappush(h, 0)       # h[0] == 0
heapq.heappop(h)           # returns 0
heapq.nlargest(2, [5, 1, 8, 3]) # [8, 5]
heapq.nsmallest(2, [5, 1, 8, 3]) # [1, 3]
```

2. Kth Largest Element in a Stream (LC 703)

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.h = nums[:]
        heapq.heapify(self.h)
        while len(self.h) > k:
            heapq.heappop(self.h)

    def add(self, val):
        heapq.heappush(self.h, val)
        if len(self.h) > self.k:
            heapq.heappop(self.h) # throw away the smallest, which can
        return self.h[0]
```


Invariant: the heap holds the k largest values seen so far, so its minimum is the k -th largest. Trace with $k = 3$, nums `[4, 5, 8, 2]`: after heapify and trimming, the heap is `{4, 5, 8}`. add(3): push 3, pop 3, return 4. add(5): push 5, pop 4, the heap is `{5, 5, 8}`, return 5. add(10): the heap becomes `{5, 8, 10}`, return 5.

3. Last Stone Weight (LC 1046): max-heap by negating

```
def lastStoneWeight(stones):
    h = [-s for s in stones]
    heapq.heapify(h)
    while len(h) > 1:
        a = -heapq.heappop(h)      # heaviest
        b = -heapq.heappop(h)      # second heaviest
        if a != b:
            heapq.heappush(h, -(a - b))
    return -h[0] if h else 0
```

Trace `[2, 7, 4, 1, 8, 1]`: 8 and 7 leave 1 → `[4, 2, 1, 1, 1]`. 4 and 2 leave 2 → `[2, 1, 1, 1]`. 2 and 1 leave 1 → `[1, 1, 1]`. 1 and 1 cancel → `[1]`. **Answer: 1.**

4. Kth Largest Element in an Array (LC 215)

```
def findKthLargest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)
    return h[0]
```

$O(n \log k)$. Alternatives: `heapq.nlargest(k, nums)[-1]`, sorting in $O(n \log n)$, or Quickselect, which averages $O(n)$ but is $O(n^2)$ in the worst case. Mentioning Quickselect scores points.

5. K Closest Points to Origin (LC 973)

Keep the k **smallest** distances, using a max-heap of size k (distances negated):

```
def kClosest(points, k):
    h = []
    for x, y in points:
        d = x * x + y * y          # compare squared distances, no sqrt
        heapq.heappush(h, (-d, x, y))
        if len(h) > k:
            heapq.heappop(h)        # removes the farthest
    return [[x, y] for _, x, y in h]
```

Or simply `heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)`.

6. Top K Frequent Elements (LC 347)

```
from collections import Counter

def topKFrequent(nums, k):
    count = Counter(nums)
    return heapq.nlargest(k, count.keys(), key=count.get)
```

$O(n \log k)$. The $O(n)$ bucket version is in `hashmap/theory.md`.

7. Merge k sorted lists (LC 23): the tie-breaker trick

```
def mergeKLists(lists):
    h = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(h, (node.val, i, node))    # i breaks ties, because
    dummy = tail = ListNode()
    while h:
        _, i, node = heapq.heappop(h)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(h, (node.next.val, i, node.next))
    return dummy.next
```

Without `i`, two equal values make Python compare the `ListNode` objects, which raises a `TypeError`. $O(N \log k)$, where N is the total number of nodes.

8. Two heaps (Find Median from Data Stream, LC 295)

Keep a max-heap for the smaller half and a min-heap for the larger half, with sizes differing by at most 1. The median comes from the tops. This is the standard hard follow-up. Know the idea even if you don't code it yet.

9. Decision table

Question	Heap setup
k-th largest, or top k largest	min-heap of size k
k-th smallest, or k closest	max-heap of size k (negate)
Repeatedly take the biggest or smallest and put back a result	max-heap or min-heap of everything
Merge k sorted sources	min-heap of the current head of each (value, index, item)
Schedule the next event by time (meeting rooms, CPU tasks)	min-heap of end times
Running median	two heaps
Only ever need the minimum once	just <code>min()</code> , $O(n)$

10. Common bugs

1. Forgetting that `heapq` is a min-heap, and getting the order backwards.
2. Negating on push but not on pop (or the reverse).
3. Pushing objects that can't be compared, without a tie-breaker.
4. Thinking `h` is sorted. Only `h[0]` is guaranteed. Use `sorted(h)` if you need the order.
5. Using a heap for top-k when the heap holds everything. That's $O(n \log n)$. Cap it at k.

11. Edge cases

- k equal to n, or $k = 1$
- Duplicate values
- An empty heap at the end (LC 1046 should return 0)
- Negative numbers when you negate for a max-heap (still works, but check your trace)

12. Interview prep

- **Purpose:** get the current extreme value fast while the data keeps changing.
 - **Control flow:** push candidates. Whenever the size goes over k , pop the one that can no longer be in the answer. Read the root.
 - **Complexity:** $O(n \log k)$ time and $O(k)$ space for top- k . $O(n)$ to heapify.
 - **What a strong candidate says:** "I keep a min-heap of the k largest values seen. Anything smaller than the root can't be in the top k , so I pop it. That's $O(n \log k)$ time and $O(k)$ space, and it works on a stream. For a one-off query on an array, Quickselect gives $O(n)$ on average."
 - **Likely follow-ups:** Task Scheduler (LC 621), Reorganize String (LC 767), Find Median from Data Stream (LC 295), Dijkstra's shortest path (a heap of `(distance, node)`).
-

Backtracking, in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved:
- Mistake and correction:

One word: Explore. **One sentence:** Build an answer one choice at a time: make a choice, go deeper, then undo the choice, so you visit every possible answer exactly once.

Executive summary

- **The decision tree:** each level is one decision. Each branch is one option. Each leaf is one complete answer.
- **The three steps:** **choose** (add to `path`), **explore** (recurse), **un-choose** (`path.pop()`).
- **Three kinds of problem:**
 - **subsets:** each element is in or out, giving 2^n answers
 - **permutations:** order matters, giving $n!$ answers
 - **combinations / combination sum:** order doesn't matter, so use a `start` index so you never look back
- **Duplicates in the input:** sort, then skip `nums[i] == nums[i-1]` at the same level of the tree.
- **Pruning:** stop a branch early when it can't lead to a valid answer. That's what makes backtracking fast enough in practice.
- **Always append** `path[:]`, a copy, never `path` itself.

1. The template

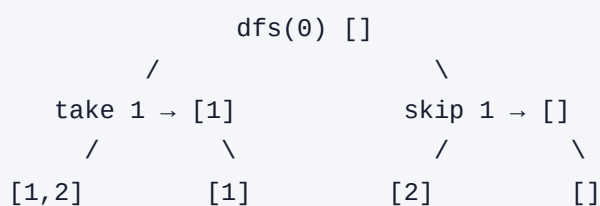
```
def backtrack_template(choices):
    res = []
    path = []
    def dfs(state):
        if is_complete(state):
            res.append(path[:])          # copy!
            return
        for choice in options(state):
            if not valid(choice):
                continue                  # prune
            path.append(choice)           # choose
            dfs(next_state(state, choice)) # explore
            path.pop()                   # un-choose
    dfs(initial_state)
    return res
```

(This is a shape to follow, not runnable code. The problems below fill it in.)

2. Subsets (LC 78): include or exclude

```
def subsets(nums):
    res, path = [], []
    def dfs(i):
        if i == len(nums):
            res.append(path[:])
            return
        path.append(nums[i])           # include nums[i]
        dfs(i + 1)
        path.pop()
        dfs(i + 1)                     # exclude nums[i]
    dfs(0)
    return res
```

Decision tree for `[1, 2]`:



Leaves in order: `[1,2], [1], [2], []`. That's $2^2 = 4$ subsets. **Why** `path[:]`: `path` is one list that keeps changing. Appending the list itself would leave `res` full of references to the same list, which is empty by the end.

3. Permutations (LC 46): track what's used

```
def permute(nums):
    res, path = [], []
    used = [False] * len(nums)
    def dfs():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            dfs()
            path.pop()
            used[i] = False          # undo BOTH changes
    dfs()
    return res
```

$n!$ permutations, each copied in $O(n)$, so $O(n \cdot n!)$.

4. Combination Sum (LC 39): reuse allowed, use a start index

```
def combinationSum(candidates, target):
    res, path = [], []
    def dfs(start, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remain:
                continue          # prune (with sorted input you could
            path.append(candidates[i])
            dfs(i, remain - candidates[i])  # i, not i + 1: the same number
            path.pop()
    dfs(0, target)
    return res
```

Why `start`: without it, `[2,3]` and `[3,2]` would both be produced. Only moving forward means each combination is produced in exactly one order.

- Each number usable **once** (LC 40): recurse with `i + 1` and skip duplicates (section 5).

5. Subsets II (LC 90): duplicates in the input

```
def subsetsWithDup(nums):
    nums.sort()                    # duplicates next to each other
    res, path = [], []
    def dfs(start):
        res.append(path[:])        # every node is a subset
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue            # same value at the same level would
            path.append(nums[i])
            dfs(i + 1)
            path.pop()
    dfs(0)
    return res
```

`[1,2,2]` gives `[], [1], [1,2], [1,2,2], [2], [2,2]`. The second `[2]` and `[1,2]` are skipped. **Why `i > start` and not `i > 0`:** you may use a second 2 deeper in the same branch (`[2,2]`). You just can't start a sibling branch with the same value.

6. Word Search (LC 79): backtracking on a grid

```
def exist(board, word):
    m, n = len(board), len(board[0])
    def dfs(r, c, k):
        if k == len(word):
            return True
        if r < 0 or r >= m or c < 0 or c >= n or board[r][c] != word[k]:
            return False
        tmp, board[r][c] = board[r][c], '#'          # mark as visited (choose)
        found = (dfs(r + 1, c, k + 1) or dfs(r - 1, c, k + 1) or
                 dfs(r, c + 1, k + 1) or dfs(r, c - 1, k + 1))
        board[r][c] = tmp                             # restore (un-choose)
        return found
    return any(dfs(r, c, 0) for r in range(m) for c in range(n))
```


7. Complexity cheat sheet

Problem	Number of answers	Time
Subsets	2^n	$O(n \cdot 2^n)$
Permutations	$n!$	$O(n \cdot n!)$
Combinations $C(n, k)$	$C(n, k)$	$O(k \cdot C(n, k))$
Grid word search	—	$O(m \cdot n \cdot 4^L)$, where L is the word length

These are exponential **because the output itself is exponential**. You can't do better than the size of the answer. Say that in the interview.

8. Decision table

Signal	Variant
"All subsets", "power set"	include/exclude, or a loop from <code>start</code>
"All orderings", "arrangements"	<code>used[]</code> array
"All combinations that sum to..."	<code>start</code> index, and <code>i</code> or <code>i+1</code> depending on reuse
Input has duplicates	sort, and skip <code>i > start and nums[i] == nums[i-1]</code>
Path in a grid	mark the cell, recurse in 4 directions, unmark
Only need the count or best , not every answer	probably DP (see <code>dp/theory.md</code>)

9. Common bugs

- `res.append(path)` instead of `path[:]`.
- Forgetting to undo the choice (`path.pop()`, `used[i] = False`, restoring the grid cell).
- `i > 0` instead of `i > start` when skipping duplicates.
- Recurring with `i + 1` when reuse is allowed, or with `i` when it isn't.
- No pruning, so the solution times out on large targets.

10. Edge cases

- Empty input: subsets should return `[]`
- Target 0, and no valid combination
- All elements equal
- A word longer than the grid has cells

11. Interview prep

- **Purpose:** list every valid configuration when the problem really needs all of them.
- **Control flow:** a recursive function with `path`. Base case: record a copy. Otherwise, loop over the options: choose, recurse, un-choose.
- **Complexity:** set by the number of answers. Say " $O(n \cdot 2^n)$, which matches the output size".
- **What a strong candidate says:** "It's a decision tree. At each index I take or skip the element. I record a copy of the path at the leaves and undo each choice on the way back up. To avoid duplicate subsets, I sort and skip equal values at the same depth."
- **Likely follow-ups:** N-Queens (LC 51), Palindrome Partitioning (LC 131), Letter Combinations of a Phone Number (LC 17), Generate Parentheses (LC 22). Also "can you just count them?", which usually means DP.
- See also `../recursion/` for your earlier recursion practice.

Graphs (BFS, DFS, topological sort), in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved:
- Mistake and correction:

One word: Traversal. **One sentence:** Visit every reachable node exactly once by keeping a `visited` set, using DFS to explore regions and BFS to find the fewest steps.

Executive summary

- **What a graph is:** nodes plus edges. Interviews give it as an edge list, an adjacency list, or a grid, where each cell connects to its 4 neighbours.
- **The key rule:** mark a node visited **when you add it** to the stack or queue, not when you pop it. Otherwise it gets added twice.
- **DFS:** recursion or a stack. Use it for connected components, flood fill and cycle detection.
- **BFS:** a queue. Gives the **shortest path in an unweighted graph** and processes things level by level ("minutes", "steps").
- **Multi-source BFS:** start with every source in the queue at once (e.g. all the rotten oranges).
- **Topological sort (Kahn's algorithm):** repeatedly take a node with in-degree 0. If you can't take all n nodes, there's a cycle. Used for course schedules and build orders.
- **Cost:** $O(V + E)$, where V is the number of nodes and E the number of edges. For an $m \times n$ grid, that's $O(m \cdot n)$.

1. Representations

```
from collections import defaultdict, deque

edges = [[0, 1], [1, 2], [2, 0]]
adj = defaultdict(list)
for a, b in edges:
    adj[a].append(b)
    adj[b].append(a)          # leave this out for a directed graph

DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))  # grid neighbours
```

2. Flood Fill (LC 733): DFS on a grid

```
def floodFill(image, sr, sc, color):
    old = image[sr][sc]
    if old == color:
        return image          # otherwise it recurses forever: filled cell
    m, n = len(image), len(image[0])
    def dfs(r, c):
        if r < 0 or r >= m or c < 0 or c >= n or image[r][c] != old:
            return
        image[r][c] = color    # recolouring IS the visited mark
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)
    dfs(sr, sc)
    return image
```

3. Number of Islands (LC 200): counting connected components

```
def numIslands(grid):
    m, n = len(grid), len(grid[0])
    count = 0
    def dfs(r, c):
        if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != '1':
            return
        grid[r][c] = '0'          # sink it = mark visited
        for dr, dc in DIRS:
            dfs(r + dr, c + dc)
    for r in range(m):
        for c in range(n):
            if grid[r][c] == '1':
                count += 1         # a new island, then sink all of it
                dfs(r, c)
    return count
```

Trace:

1 1 0	the first '1' at (0,0) sinks (0,0),(0,1),(1,0) → count 1
1 0 0	the next unvisited '1' is (2,2) → count 2
0 0 1	

Invariant: every land cell already seen is '0', so each island is counted once. Max Area of Island (LC 695) is the same, except `dfs` returns `1 + sum of neighbours`.

4. BFS for the fewest steps: Rotting Oranges (LC 994), multi-source

```
def orangesRotting(grid):
    m, n = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 2:
                q.append((r, c))          # every rotten orange starts
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    while q and fresh:
        for _ in range(len(q)):          # one level = one minute
            r, c = q.popleft()
            for dr, dc in DIRS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and grid[nr][nc] == 1:
                    grid[nr][nc] = 2      # mark when enqueueing
                    fresh -= 1
                    q.append((nr, nc))
            minutes += 1
    return -1 if fresh else minutes
```

Trace `[[2,1,1],[1,1,0],[0,1,1]]`: minute 1 rots (0,1) and (1,0). Minute 2 rots (0,2) and (1,1). Minute 3 rots (2,1). Minute 4 rots (2,2). **Answer: 4. Why BFS gives the shortest path:** it finishes every node at distance d before touching distance $d+1$, so the first time you reach a node is by the shortest route.

5. Plain BFS shortest path on a graph

```
def shortest_path(adj, src, dst):
    dist = {src: 0}
    q = deque([src])
    while q:
        u = q.popleft()
        if u == dst:
            return dist[u]
        for v in adj[u]:
            if v not in dist:          # dist doubles as the visited set
                dist[v] = dist[u] + 1
                q.append(v)
    return -1
```

6. Course Schedule (LC 207): topological sort with Kahn's algorithm

[a, b] means "to take a, you must first take b", so the edge goes $b \rightarrow a$.

```
def canFinish(numCourses, prerequisites):
    adj = [[] for _ in range(numCourses)]
    indeg = [0] * numCourses
    for a, b in prerequisites:
        adj[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(numCourses) if indeg[i] == 0)
    done = 0
    while q:
        u = q.popleft()
        done += 1
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:          # all of v's prerequisites are done
                q.append(v)
    return done == numCourses        # anything left over is stuck in a cycle
```

- Course Schedule II (LC 210): return the order in which nodes were popped.
- **DFS alternative:** give each node 3 states (unvisited, visiting, done). Reaching a "visiting" node means there's a cycle.

7. Clone Graph (LC 133): a map from each original node to its copy

```
def cloneGraph(node):
    if not node:
        return None
    copies = {node: Node(node.val)}
    q = deque([node])
    while q:
        u = q.popleft()
        for v in u.neighbors:
            if v not in copies:
                copies[v] = Node(v.val)
                q.append(v)
            copies[u].neighbors.append(copies[v])
    return copies[node]
```

8. Decision table

Question	Tool
How many regions or islands?	DFS or BFS from each unvisited cell
Fewest steps or minutes (unweighted)	BFS, level by level
Spreading from many sources at once	Multi-source BFS
Order with dependencies, "is it possible?"	Topological sort (Kahn)
Cycle in a directed graph	Kahn (done < n) or DFS with 3 states
Shortest path with weights	Dijkstra (heap), outside this roadmap
Dynamic connectivity, "are these connected?"	Union-find, outside this roadmap

9. Common bugs

1. Marking visited when popping instead of when pushing, so nodes get enqueued many times.
2. Flood fill when `old == color`, which recurses forever.
3. Forgetting bounds checks, or checking them after indexing into the grid.
4. Getting the edge direction backwards in Course Schedule.
5. Recursion depth on a 300×300 grid (90,000 cells). Use BFS or an explicit stack.

6. Counting minutes one too many (check `while q and fresh`).

10. Edge cases

- An empty grid, or a 1×1 grid
- No land, or all land
- A disconnected graph (loop over every node, not just node 0)
- No fresh oranges at the start (answer 0)
- Self-loops and duplicate edges

11. Interview prep

- **Purpose:** explore connectivity and shortest distances in networks, grids and dependency lists.
- **Control flow:** build the adjacency list. For each unvisited node, start a traversal. Mark visited when adding, and add the unvisited neighbours.
- **Complexity:** $O(V + E)$ time, $O(V)$ space for the visited set and the queue or stack.
- **What a strong candidate says:** "I treat the grid as a graph where each cell has 4 edges. I scan every cell, and each unvisited land cell starts a new island, which I sink with DFS so it isn't counted again. Every cell is processed a constant number of times, so it's $O(m \cdot n)$."
- **Likely follow-ups:** Pacific Atlantic Water Flow (LC 417), Surrounded Regions (LC 130), Word Ladder (LC 127, BFS), Network Delay Time (LC 743, Dijkstra), Redundant Connection (LC 684, union-find).

Dynamic programming (1-D), in full

Pattern card (fill from memory, don't copy from below)

- Trigger:
- Invariant:
- Template/pseudocode from memory:
- Complexity:
- Solved/re-solved: 53 (Kadane, in ../array)
- Mistake and correction:

One word: Reuse. **One sentence:** When the answer for a big input is built from answers to smaller inputs, and those smaller answers repeat, compute each one once and store it.

Executive summary

- **Two conditions for DP:**
 1. **Optimal substructure:** the best answer is built from the best answers to smaller pieces.
 2. **Overlapping subproblems:** the same smaller piece is needed again and again.
- **The four questions** (answer them in writing before coding):
 1. **State:** what does `dp[i]` mean, in one sentence?
 2. **Transition:** how is `dp[i]` computed from smaller states?
 3. **Base cases:** the smallest states, known directly.
 4. **Order and answer:** which direction to fill in, and which entry is the final answer.
- **Top-down:** recursion plus `@cache` (memoization). Easiest to write from the recursive idea.
- **Bottom-up:** a loop filling a table. No recursion limit, and often reducible to $O(1)$ space by keeping only the last few values.

- **Backtracking vs DP:** backtracking lists every answer. DP counts them or finds the best one, without listing them.

1. Climbing Stairs (LC 70): the first DP

State: `ways(i)` = the number of ways to reach step i . **Transition:** the last move was 1 step or 2 steps, so `ways(i) = ways(i-1) + ways(i-2)`. **Base cases:** `ways(0) = 1` (standing still), `ways(1) = 1`.

Top-down:

```
from functools import cache

def climbStairs_memo(n):
    @cache
    def ways(i):
        if i <= 1:
            return 1
        return ways(i - 1) + ways(i - 2)
    return ways(n)
```

Bottom-up, $O(1)$ space:

```
def climbStairs(n):
    a, b = 1, 1                # ways(i-2), ways(i-1)
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

Check by hand: $n = 1$ gives 1. $n = 2$ gives 2 (1+1, or 2). $n = 3$ gives 3. $n = 4$ gives 5. **Why memoization matters:** plain recursion calls `ways(n-2)` from two places, `ways(n-3)` from three, and so on. That's $O(2^n)$. Caching makes it $O(n)$.

2. Min Cost Climbing Stairs (LC 746)

State: `dp[i]` = the minimum cost to stand on step i , where the top is step n . **Transition:** `dp[i] = min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2])`. **Base cases:** `dp[0] = dp[1] = 0`, because you may start on either step.

```
def minCostClimbingStairs(cost):
    a = b = 0 # dp[i-2], dp[i-1]
    for i in range(2, len(cost) + 1):
        a, b = b, min(b + cost[i - 1], a + cost[i - 2])
    return b
```

Trace `[10, 15, 20]`: $dp[2] = \min(0+15, 0+10) = 10$. $dp[3] = \min(10+20, 0+15) = 15$.

3. House Robber (LC 198): take or skip

State: `dp[i]` = the most money from houses `0..i`. **Transition:** skip house `i` (`dp[i-1]`) or rob it (`dp[i-2] + nums[i]`). Take the larger.

```
def rob(nums):
    prev, cur = 0, 0 # best up to i-2, best up to i-1
    for x in nums:
        prev, cur = cur, max(cur, prev + x)
    return cur
```

Trace `[2, 7, 9, 3, 1]`:

x	prev (i-2)	cur (i-1)	new cur = max(cur, prev + x)
2	0	0	2
7	0	2	7
9	2	7	11
3	7	11	11
1	11	11	12

Houses in a circle (LC 213): run it on `nums[1:]` and on `nums[:-1]`, and take the larger.

4. Coin Change (LC 322): the state is an amount, not an index

State: `dp[a]` = the fewest coins that make amount `a`. **Transition:** `dp[a] = min(dp[a - c] + 1)` over every coin `c ≤ a`. **Base case:** `dp[0] = 0`. Everything else starts as "impossible" (use `amount + 1` for that).

```
def coinChange(coins, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

Coins `[1, 2, 5]`, amount 11: $dp[5] = 1$, $dp[10] = 2$, $dp[11] = dp[10] + 1 = 3$ ($5 + 5 + 1$). $O(\text{amount} \times \text{number of coins})$. **Why greedy fails:** with coins `[1, 3, 4]` and amount 6, greedy picks $4 + 1 + 1$ (3 coins), but $3 + 3$ uses 2.

5. Longest Increasing Subsequence (LC 300)

State: `dp[i]` = the length of the longest increasing subsequence that **ends at** index i .

Transition: `dp[i] = 1 + max(dp[j])` over every $j < i$ with `nums[j] < nums[i]`. **Answer:** `max(dp)`, not `dp[-1]`.

```
def lengthOfLIS(nums):
    dp = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

$O(n^2)$. The $O(n \log n)$ version keeps `tails[k]` = the smallest possible last value of an increasing subsequence of length $k+1$, and places each number with `bisect_left`. That's where DP and binary search meet (see `../binarySearch/theory.md`).

6. Maximum Subarray (LC 53): Kadane's algorithm is DP

State: `best_ending_here` = the best sum of a subarray that ends at i . **Transition:** `max(x, best_ending_here + x)`, meaning start fresh here or extend the previous one.

```
def maxSubArray(nums):
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

You already solved it in `../array/53_maximum_subarray.py`. Now you know it's DP.

7. How to find the state (when you're stuck)

1. Write the brute-force recursion first: "the answer for i uses the answers for...".
2. List what changes between recursive calls. Those values are the state. One changing value means 1-D DP.
3. Check whether any call repeats. If yes, cache it.
4. Turn it into a loop in the order the dependencies point (small i to large i).
5. If `dp[i]` only uses `dp[i-1]` and `dp[i-2]`, keep two variables instead of the whole array.

8. Decision table

Signal	State
"Number of ways to reach..."	<code>dp[i]</code> = ways to reach i , summed over the last moves
"Min or max cost to reach..."	<code>dp[i]</code> = best cost to reach i , min/max over the last moves
"Can't pick adjacent items"	take or skip: <code>max(dp[i-1], dp[i-2] + x)</code>
"Fewest items to make a total"	<code>dp[amount]</code> over each item
"Longest subsequence with a property"	<code>dp[i]</code> = best that ends at i , answer = <code>max(dp)</code>
"Best contiguous subarray"	Kadane: best ending here
"List all the solutions"	not DP: backtracking

9. Common bugs

1. A vague state. If you can't say what `dp[i]` means in one sentence, stop and define it.

2. Wrong base cases (`ways(0) = 1` , not 0).
3. Returning `dp[-1]` when the answer is `max(dp)` (LIS).
4. Using 0 as "impossible" in a min DP. Use infinity or `amount + 1`.
5. Recursion without a cache, which is exponential time.
6. Recursion depth errors with `@cache` for large n. Switch to bottom-up.

10. Edge cases

- $n = 0$, and $n = 1$
- Amount 0 (answer 0), and an amount that can't be made (answer -1)
- All negative numbers (Kadane must return the largest single element)
- A single house

11. Interview prep

- **Purpose:** turn exponential recursion into polynomial time by solving each subproblem once.
- **Control flow:** define the state, write the transition, set the base cases, fill in the table in dependency order, return the answer entry.
- **Complexity:** number of states $\times$ work per state. For example, Coin Change is $O(\text{amount} \times \text{coins})$, and LIS is $O(n^2)$.
- **What a strong candidate says:** "Let `dp[i]` be the most money from the first i houses. For house i , I either skip it and take `dp[i-1]`, or rob it and take `dp[i-2]` plus its value. That's $O(n)$, and since only two previous values are used, I can keep two variables for $O(1)$ space."
- **Likely follow-ups:** Decode Ways (LC 91), Word Break (LC 139), Partition Equal Subset Sum (LC 416), Longest Palindromic Substring (LC 5), then 2-D DP (Unique Paths LC 62, Longest Common Subsequence LC 1143).